



MAY 3, 2026

OC:SE FORMATIVE ASSESSMENT 2

TEST PLAN

NALEDI MOFOKENG
20240778



Table of Contents

Testing Objectives	2
System Purpose	2
Testing Goals	2
Scope of testing.....	3
 Testing Types Identification	 3
Testing Approaches	3
 Testing Environment Setup	 4
Hardware	4
Software	4
Tools used.....	4

1.1 Testing Objectives

System Purpose:

The Umoja Learning Management System (ULMS) is a cloud-enabled desktop application built in C# (WinForms) designed to provide a centralized platform to manage the full academic lifecycle of students within the Umoja Group.

The system provides five core functions: student registration, login authentication, course enrolment, marks capture, and report generation.

The primary business goal is to provide a seamless and accurate experience for both students and administrators. Academic administrators will be able to register and manage students, assign them to courses, capture their assessment marks, calculate averages, and generate structured reports – all within a secure, validated environment. And students will be able to have a self-service registration, secure access to their profiles and track their academic progress – all within a secure, validated environment.

Testing Goals:

- Confirm that all functional requirements work correctly according to the specified business requirements.
- Identify and document all defects in the source code, including logic errors, missing validations, and incorrect calculations.
- Confirm that the login authentication correctly validates credentials and prevents unauthorised access.
- Validate that marks are captured within acceptable numeric ranges (0–100) and that averages are calculated correctly to avoid inconsistencies.
- Ensure course enrolment rules enforce no duplicate registrations for the same student and maintain data integrity.
- Confirm that reports are generated accurately and within acceptable performance thresholds.
- Provide a formal quality assurance decision on whether the system is ready for production deployment.

Scope of testing:

In scope: Explicitly mentioned items needing attention

Core Functional Modules:

- Student Registration module – input validation and data persistence
- Login Authentication module – credential validation, access control
- Course Enrolment module – enrolment rules, duplication prevention
- Marks Capture module – numeric validation, boundary conditions
- Error handling and exception management across all modules

Logic & Calculations:

- Average Calculation logic – mathematical correctness

Performance Optimization:

- Report Generation module – output accuracy, loading performance

Documentation:

- Missing testing documentation and final QA report

Out of scope: Items excluded from this specific testing phase

- Network infrastructure – testing the internet speed
 - Cloud synchronisation – testing internal server maintenance of Azure or AWS
 - Third-party integrations not present in the provided source code
-

1.2 Testing Types Identification

Testing Type	Example	Testing approach
Functional Testing	Login validation	Create test cases to verify that valid credentials grant access and invalid ones are rejected with appropriate feedback.
Input Validation	Marks entry	Enter boundary values to ensure the system only accepts numerical values within a specific range (0–100) and rejects symbols or text correctly.
Integration Testing	Course allocation	Test that when a student enrolls, the record is correctly linked between the Student table and the Course table in the database.
Performance Testing	Report loading	Measure the time taken to generate a full student body report under both normal and peak conditions; optimize SQL queries if loading exceeds 3 seconds.
Usability Testing	Navigation	Check if the menu layout is intuitive and check the number of steps it takes to complete the core tasks.

Security Testing	Authentication	Ensure passwords are encrypted/hashed and that a user cannot bypass the login screen by manually typing a URL (Session Management).
------------------	----------------	---

1.3 Testing Environment Steup

Here is the specific configuration of hardware and software where the testing team will execute the test cases.

Hardware:

- Processor: Intel Core i5 or equivalent (provides multi-threading power)
- RAM: 8GB (minimum to handle Visual Studio and SQL Server instances simultaneously)
- Storage: 50 GB available disk space (SSD rather than HDD to improve the test execution speed)
- Display: 1920 × 1080 resolution (enables you to see the entire dashboard)
- Input devices: Standard keyboard and mouse
- Connectivity: Stable LAN or Wi-Fi

Software:

- Operating System: Windows 10 / Windows 11 (64-bit)
- IDE: Visual Studio (Community or Professional)
- .NET Framework: .NET 6.0 / .NET Framework 4.8
- Database Engine: SQL Server 2019 Express or SQL Server LocalDB
- Database Management: SSMS (SQL Server Management Studio)
- Version Control: Git / GitHub

Testing Tools Used:

- Visual Studio 2022: IDE for running, debugging and stepping through source code (Dynamic testing)
- SSMS (SQL Server Management Studio) : Database query validation and data verification (Data testing)
- GitHub: Source code repository and version control (Static testing)
- Test data sets: Prepared CSV / manual data covering valid, invalid and boundary scenarios (Data testing)
- Stopwatch / Performance Profiler (VS Diagnostic Tools): Report loading time measurement (Efficiency testing)